



Cross over of Genetics Algorithms

Agenda

01

**Chromosome Representation
Representation**

03

Binary to Gray Code

05

One-Point Crossover

07

Uniform Crossover

02

**Binary Coding Disadvantage
Disadvantage**

04

Crossover (Recombination)

06

Two-Point Crossover

08

Arithmetic Crossover



CHROMOSOME REPRESENTATION

Definition: A chromosome is a string of genes that represents a solution to an optimization problem. Each gene encodes a part of the solution.

Example:

We want find the best values of: x and y for optimizing a function $f(x,y)$ where $x,y \in [0,31]$

- Chromosome: `[10110|01101]`
- First 5 bits represent $x = 22$
- Last 5 bits represent $y = 13$
- Solution: $(x=22, y=13)$



CHROMOSOME REPRESENTATIONS

Common Representation Types:

1

Binary Strings (0s and 1s)

A chromosome made of bits: 0 or 1 each bit is a gene.
Good for: Discrete decisions, Yes / No problems, On / Off variables And Classical Genetic Algorithm

Example: 10110101

2

Real-Valued Vectors

A chromosome made of real numbers (decimals).
Each gene is a continuous value.

Maintains precision

Good for :ML parameters and continuous variable

Example: [2.5, 3.1, 1.8, 4.2]

3

Permutations

A chromosome that stores an order / arrangement of items. Every item appears once.

Used when sequence matters.

Good for: Routing, Scheduling and Visiting order
Traveling Salesman Problem (TSP)

Example: [3, 1, 4, 2, 5]

4

Trees

A chromosome stored as a tree structure, not a line.

Used when solution is a formula or program

Good for: Symbolic regression and
Genetic programming

Example: (+ (* x 2) (- y 3))

BINARY CODING DISADVANTAGE

Definition: Binary coding represents solutions as strings of 0s and 1s. While simple, it has important limitations in genetic algorithms.

01

Hamming Cliff Problem

- Small solution changes can require many bit flips

02

Inefficient Search

- GA may break good solutions while trying improve them.
- Example:
Good solution : 01111111
Mutation flip one bit : 00111111
- A small bit change can destroy a good answer. So search becomes messy.

03

Representation Inefficiency

- Continuous variables need many bits
- Chromosomes grow quickly for larger problems

Example Problem:

For $x \in [0, 255]$ with 8-bit binary: $127 = 01111111$, $128 = 10000000$ — only 1 unit apart, but 8 bits different.

BINARY TO GRAY CODE

Definition: Gray Code is an encoding where consecutive values differ by exactly one bit, helping solve the Hamming Cliff problem.

One-bit changes

Smoother search

Better for optimization

Conversion

Binary → Gray: $G[i] = B[i] \text{ XOR } B[i+1]$

Gray → Binary: $B[i] = G[i] \text{ XOR } B[i+1]$

BINARY TO GRAY CODE - EXAMPLE & PSEUDOCODE

PSEUDOCODE

Example Conversion Table:

Decimal	Binary	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110

 **Key Observation:** 7→8 in Gray Code: 0100→1100
(only 1 bit changes!)

Pseudocode:

```
FUNCTION BinaryToGray(binary):  
    gray = binary[0] // First bit stays same  
    FOR i = 1 TO length(binary)-1:  
        gray[i] = binary[i-1] XOR binary[i]  
    RETURN gray
```

CROSSOVER (RECOMBINATION)

Definition: Crossover is a genetic operator that combines genetic material from two parent chromosomes to create offspring.

Purpose:

- Combine good traits from both parents

- Create new solutions by mixing existing ones

- Maintain genetic diversity

General Crossover Process:

```
FUNCTION Crossover(parent1, parent2, crossoverRate):  
    IF random() < crossoverRate:  
        offspring1, offspring2 = PerformCrossover(parent1, parent2)  
        RETURN offspring1, offspring2  
    ELSE:  
        RETURN parent1, parent2
```

ONE-POINT CROSSOVER

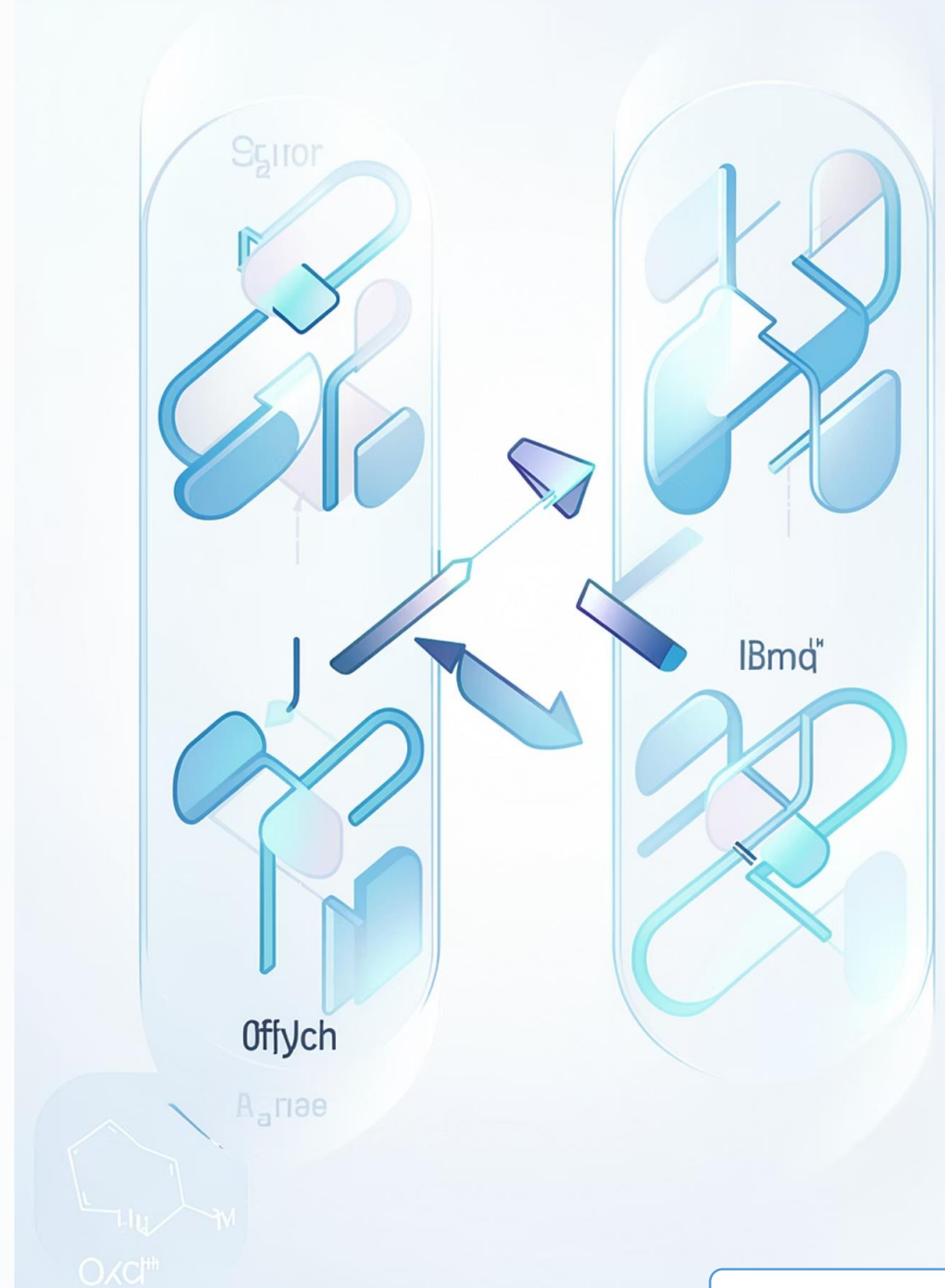
Definition: One-point crossover selects a single random point in the chromosome and exchanges all genes after that point between two parents.

How It Works:

- Select a random crossover point (1 to L-1, where L is chromosome length)
- Split both parents at this point
- Swap the segments after the crossover point
- Create two offspring

One-Point genetic

One point genilisi cralawosla



ONE-POINT Crossover - EXAMPLE & PSEUDOCODE

Example:

```
Parent 1: 1 1 0 1 | 0 1 1 0
Parent 2: 0 0 1 0 | 1 1 0 1
           (crossover point at position 4)
```

```
Offspring 1: 1 1 0 1 | 1 1 0 1
Offspring 2: 0 0 1 0 | 0 1 1 0
```

Pseudocode:

```
FUNCTION OnePointCrossover(parent1, parent2):
    L = length(parent1)
    crossoverPoint = random(1, L-1)

    offspring1 = parent1[0:crossoverPoint] +
parent2[crossoverPoint:L]
    offspring2 = parent2[0:crossoverPoint] +
parent1[crossoverPoint:L]

    RETURN offspring1, offspring2
```

TWO-POINT CROSSOVER

Definition: Two-point crossover selects two random points in the chromosome and exchanges the segment between these points between two parents.

How It Works:

1. Select two random crossover points ($p1$ and $p2$, where $p1 < p2$)
2. Split both parents at these points
3. Swap the middle segment between the two parents
4. Create two offspring

TWO-POINT CROSSOVER - EXAMPLE & PSEUDOCODE

PSEUDOCODE

Example:

```
Parent 1: 1 1 | 0 1 0 1 | 1 0
Parent 2: 0 0 | 1 0 1 1 | 0 1
          (crossover points at positions 2 and
          6)
```

```
Offspring 1: 1 1 | 1 0 1 1 | 1 0
Offspring 2: 0 0 | 0 1 0 1 | 0 1
```

Pseudocode:

```
FUNCTION TwoPointCrossover(parent1, parent2):
    L = length(parent1)
    p1 = random(1, L-2)
    p2 = random(p1+1, L-1)

    offspring1 = parent1[0:p1] + parent2[p1:p2] +
parent1[p2:L]
    offspring2 = parent2[0:p1] + parent1[p1:p2] +
parent2[p2:L]

    RETURN offspring1, offspring2
```

TWO-POINT CROSSOVER - ADVANTAGES & DISADVANTAGES

Advantages:

Better genetic mixing than one-point

Reduces positional bias (genes in middle are more likely to be separated)

More exploration of solution space

Commonly used in practice

Disadvantages:

Slightly more complex than one-point

Still may not mix genes optimally

Computational cost slightly higher

UNIFORM CROSSOVER

Definition: Uniform crossover treats each gene independently, selecting each gene from either parent with equal probability.

How It Works:

- Choose two parent chromosomes
- For each gene, flip a fair coin
- Assign the gene from Parent A or Parent B
- Repeat across the entire chromosome

Advantages:

- Maximum genetic mixing
- No positional bias

Disadvantages:

- May disrupt good building blocks

UNIFORM CROSSOVER - EXAMPLE & PSEUDOCODE

PSEUDOCODE

Example:

Parent 1: 1 1 0 1 0 1 1 0

Parent 2: 0 0 1 0 1 1 0 1

Mask: 0 1 0 1 1 0 1 0

Offspring 1: 0 1 1 1 0 1 1 1 (0's take Parent 2 while 1's take Parent 1)

Offspring 2: 1 0 0 0 1 1 0 0 (0's take Parent 1 while 1's take Parent 2)

Pseudocode:

```
FUNCTION UniformCrossover(parent1, parent2, Pu=0.5):  
    offspring1 = []  
    offspring2 = []  
    FOR i = 0 TO length(parent1)-1:  
        IF random() < Pu:  
            offspring1[i] = parent1[i]  
            offspring2[i] = parent2[i]  
        ELSE:  
            offspring1[i] = parent2[i]  
            offspring2[i] = parent1[i]  
    RETURN offspring1, offspring2
```

ARITHMETIC CROSSOVER

Definition: Arithmetic crossover applies arithmetic operations to parent chromosomes to create offspring, for real-valued representations.

How It Works:

- Select two parents with real-valued genes
- Apply arithmetic operations to corresponding genes
- Create offspring using weighted combinations of parent genes

Types of Arithmetic Crossover:

1. Intermediate Crossover:

```
offspring1[i] =  $\alpha$  × parent1[i] + (1- $\alpha$ ) × parent2[i]  
offspring2[i] =  $\alpha$  × parent2[i] + (1- $\alpha$ ) × parent1[i]  
where  $\alpha \in [0,1]$  (typically 0.5 for average)
```

2. Line Crossover:

```
offspring1[i] = parent1[i] +  $\alpha$  × (parent2[i] - parent1[i])  
offspring2[i] = parent2[i] +  $\alpha$  × (parent1[i] - parent2[i])  
where  $\alpha > 1$  (explores beyond parent values)
```

ARITHMETIC CROSSOVER

When to Use:

- Real-valued optimization problems
- Continuous parameter spaces
- When smooth interpolation is beneficial
- Function optimization (e.g., $f(x,y) = x^2 + y^2$)
- Neural network weight optimization

Example:

Parent 1: [2.5, 3.1, 1.8]

Parent 2: [4.2, 2.3, 3.5]

$\alpha = 0.5$ (intermediate crossover)

Offspring 1: $[0.5 \times 2.5 + 0.5 \times 4.2, 0.5 \times 3.1 + 0.5 \times 2.3, 0.5 \times 1.8 + 0.5 \times 3.5]$
 $= [3.35, 2.7, 2.65]$

Offspring 2: $[0.5 \times 4.2 + 0.5 \times 2.5, 0.5 \times 2.3 + 0.5 \times 3.1, 0.5 \times 3.5 + 0.5 \times 1.8]$
 $= [3.35, 2.7, 2.65]$

Thank You for Your Attention